Day 113: The Kernel Trick From Scratch — a Real RBF-Kernel Demonstration

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 17, Day 2 | Complete study guide

Watch: Support Vector Machines Part 2: The Polynomial Kernel (StatQuest) | Support Vector Machines Part 3: The Radial (RBF) Kernel (StatQuest)

Script: `day113_kernel_trick.py` · every number in this guide is copied by program from `day113_run_output.txt`, the real output of that script.

Day 112's solver only ever reads a table of dot products. Replace the table with a kernel and the same solver draws curved boundaries. The trick is checked exactly: a degree-2 polynomial kernel equals the dot products of 2145 explicit features to 5.7e-14. On handwritten digits an RBF kernel beats a straight boundary on all 5 splits (+0.0182), and a tuned one reaches 0.9870. On breast cancer the gain is small (+0.0058). Random features approximate the kernel well enough to come within 0.003 of it at 300 dimensions and match it from 1,000.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

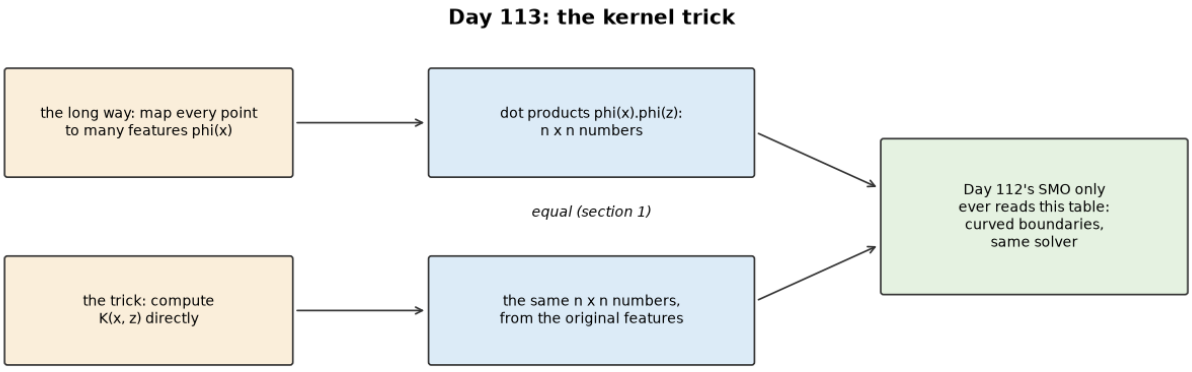
A **kernel** K(x, z) is a similarity between two points that equals a dot product $\phi(x)\cdot\phi(z)$ after some mapping ϕ into new features. The **kernel trick** computes K directly, without ever building $\phi(x)$, which may have thousands of features or infinitely many.

The **RBF** (radial basis function, or Gaussian) kernel, $\exp(-\gamma|x - z|^2)$, is 1 for identical points and falls towards 0 with distance. γ sets how fast: large γ means only very close points count as similar.

REAL-WORLD SCENARIO

Judging whether two songs are alike without writing down every feature of each: you compare them directly. A kernel is that direct comparison, and a straight boundary in the hidden feature space is a curved one in the original.

SYSTEM DIAGRAM



The long way and the trick give the same table, and Day 112's solver only reads the table.

Today's part for the week's build

`day113_exercise.py` is the kernels and random features, each returning what Day 112's solver needs. Its real output:

ACTUAL OUTPUT

```
two features become 6: 1, sqrt2 x1, sqrt2 x2, x1^2, x2^2, sqrt2 x1 x2
(x.z + 1)^2 against phi(x).phi(z): largest gap 1.1e-15
by hand for the first two points: x.z = 1*0 + 2*(-1) = -2, so (x.z + 1)^2 = 1; kernel 1.0
RBF, gamma 0.5: diagonal [1.0, 1.0, 1.0, 1.0], symmetric True, smallest eigenvalue 0.9555
by hand: |x1 - x2|^2 = 1 + 9 = 10, exp(-0.5 * 10) = 0.006738; kernel 0.006738
random features D =    10: largest error against the exact kernel 0.5834
random features D =   100: largest error against the exact kernel 0.2266
random features D =  1000: largest error against the exact kernel 0.0505
random features D = 10000: largest error against the exact kernel 0.0300
```

Data: breast cancer and handwritten digits (1,797 images of 8 × 8 pixels), bundled with scikit-learn. Day 112's SMO is imported unchanged; scikit-learn's SVC is run once as an outside check.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The trick, checked

DEFINITION — start here, no prior knowledge assumed

For two features, $(x \cdot z + 1)^2$ expands into the dot product of six features: 1, $\sqrt{2}x_1$, $\sqrt{2}x_2$, x_1^2, x_2^2, $\sqrt{2}x_1x_2$. For 64 pixels, it is 2,145 features.

REAL-WORLD SCENARIO

Multiplying two long numbers either digit by digit, or with a formula that gives the same answer in one step.

MATHEMATICS

$$(x \cdot z + 1)^2 = \phi(x) \cdot \phi(z), \quad \phi(x) = (1, \sqrt{2}x_i, x_i^2, \sqrt{2}x_i x_j \text{ for } i < j)$$

$$K_{RBF}(x, z) = \exp(-\gamma\, |x - z|^2)$$

IMPLEMENTATION

```
def linear(A, B):
    return A @ B.T
def poly(A, B, degree=2, c=1.0):
    return (A @ B.T + c) ** degree
def sqdist(A, B):
    return (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
def rbf(A, B, gamma):
```

```
    return np.exp(-gamma * np.maximum(sqdist(A, B), 0))
def phi_poly2(X):
    n, p = X.shape; cols = [np.ones(n)] + [np.sqrt(2) * X[:, i] for i in range(p)]
    cols += [X[:, i] ** 2 for i in range(p)]
    cols += [np.sqrt(2) * X[:, i] * X[:, j] for i in range(p) for j in range(i + 1, p)]
    return np.column_stack(cols)
def rff(X, D, gamma, seed):
    rng = np.random.RandomState(seed)
    W = rng.normal(scale=np.sqrt(2 * gamma), size=(X.shape[1], D))
    c = rng.uniform(0, 2 * np.pi, D)
    return lambda Z: np.sqrt(2.0 / D) * np.cos(Z @ W + c)
```

ACTUAL OUTPUT

```
64 pixels -> 2145 explicit features for (x.z + 1)^2; largest gap between the two 500 x 500 matrices 5.7e-14
    (values up to 520)
time: explicit features then dot products 22.9 ms; the kernel 2.7 ms (one run, a rough comparison)
  degree 2: the explicit map needs 2,145 features; the kernel needs 64
  degree 3: the explicit map needs 47,905 features; the kernel needs 64
  degree 5: the explicit map needs 11,238,513 features; the kernel needs 64
the RBF kernel corresponds to infinitely many features, so only the kernel form can be computed exactly
Actual output from this lesson's run.
```

What it means

The two 500 × 500 tables agree to 5.7e-14, rounding error on values up to 520. The cost is the point: the explicit route needs 2,145 features for degree 2, 47,905 for degree 3 and 11,238,513 for degree 5, while the kernel always works with the original 64. For the RBF kernel the hidden feature space is infinite, so the trick is the only way.

WHY CHOOSE THIS?

Rich boundaries at the cost of the original features.

WHY NOT THIS? (TRADEOFFS)

The kernel table is n × n: fine for thousands of rows, impossible for millions.

2. What makes a valid kernel

DEFINITION — start here, no prior knowledge assumed

A function counts as a kernel only if every table it produces is **positive semi-definite**: no negative eigenvalues. Then it is a table of real dot products in some feature space, and the SVM's problem has one best answer.

REAL-WORLD SCENARIO

A distance chart where some 'distances' are negative cannot come from any real map.

ACTUAL OUTPUT

```
  linear                              : smallest eigenvalue -1.093e-12, largest +3.203e+03,
      negative ones 0
  polynomial, degree 2                : smallest eigenvalue +2.823e-01, largest +4.298e+04,
      negative ones 0
  RBF, gamma 0.03                     : smallest eigenvalue +7.107e-04, largest +2.277e+02,
      negative ones 0
  tanh(x.z - 1), a popular non-kernel  : smallest eigenvalue -7.054e-03, largest +3.000e+02,
      negative ones 62
  minus the distance |x - z|          : smallest eigenvalue -9.036e+02, largest +8.344e+01,
      negative ones 1
a kernel matrix must be positive semi-definite (no negative eigenvalues): then it is a table of
    real dot products, and the dual problem has one best answer
Actual output from this lesson's run.
```

What it means

Linear, polynomial and RBF tables have no negative eigenvalues (the linear one's -1.093e-12 is rounding). Two popular-looking similarities fail: tanh(x·z – 1), once widely used as the 'sigmoid kernel', has 62 negative eigenvalues on these 300 images, and minus the distance has one large one. With such a table the SVM's problem may have no single best answer.

WHY CHOOSE THIS?

A cheap test anyone can run before trusting a new similarity.

WHY NOT THIS? (TRADEOFFS)

It checks one sample of points, not every possible one.

3. A real curved boundary

DEFINITION — start here, no prior knowledge assumed

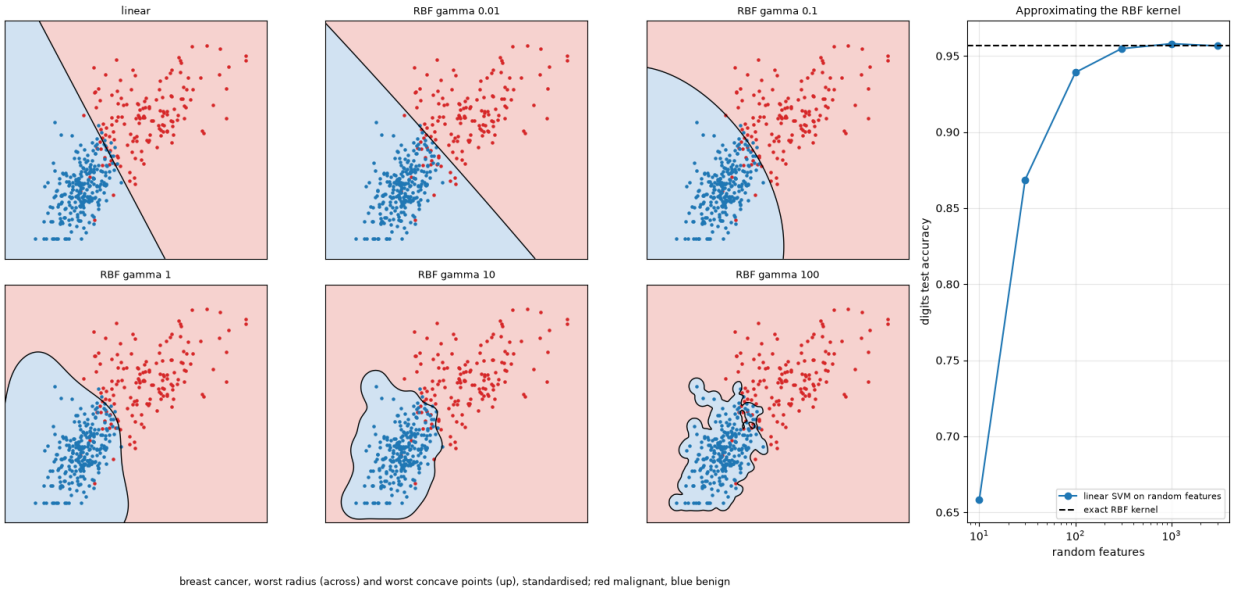
Two breast-cancer measurements, so that the boundary can be drawn. γ goes from 0.01 (each point influences a wide area) to 100 (each point influences only its immediate neighbourhood).

REAL-WORLD SCENARIO

Drawing a coastline with a thick brush (smooth, misses the inlets) or a fine pen (follows every rock).

ACTUAL OUTPUT

```
398 training, 171 test tumours; 'worst radius' and 'worst concave points', standardised; C = 1
kernel              support  train acc  test acc
linear                   65     0.9447    0.9591
RBF gamma 0.01          140     0.9347    0.9591
RBF gamma 0.1            81     0.9472    0.9532
RBF gamma 1              74     0.9497    0.9532
RBF gamma 10            178     0.9573    0.9591
RBF gamma 100           328     0.9749    0.9474
outside check, RBF gamma 1: test predictions that agree with scikit-learn's SVC 0.9942
Actual output from this lesson's run.
```



Left: the boundary for each kernel on two breast-cancer measurements. Right: digits, a linear SVM on random Fourier features against the exact RBF kernel.

What it means

The picture shows the whole story of γ. At 0.01 the boundary is almost straight; at 1 it bends around the benign cluster; at 100 it wraps individual tumours, training accuracy rises to 0.9749 and test falls to 0.9474, with 328 of 398 points as support vectors. On these two features a straight line already does as well as anything (0.9591): the data does not need a curve. The from-scratch RBF agrees with scikit-learn's SVC on 0.9942 of test tumours, one tumour different at the solver's tolerance.

WHY CHOOSE THIS?

γ gives direct control over how local the boundary is.

WHY NOT THIS? (TRADEOFFS)

Large γ memorises; the number of support vectors is a warning sign.

4. Where a kernel helps

DEFINITION — start here, no prior knowledge assumed

Three kernels on two problems, 5 splits each, with C = 1 and a common default for γ. Digits has ten classes, handled by one-vs-rest: one SVM per digit, the most confident one wins.

REAL-WORLD SCENARIO

A curved ruler helps with circles, not with straight lines.

IMPLEMENTATION

```
def ovr_fit(K, y, C):
    return [fit_kernel_svm(K, np.where(y == c, 1.0, -1.0), C)[:2] for c in range(int(y.max()) +
        1)]
def ovr_predict(models, y, K_new):
    scores = [K_new.T @ (a * np.where(y == c, 1.0, -1.0)) + b for c, (a, b) in enumerate(models)]
    return np.argmax(np.stack(scores, 1), 1)
```

ACTUAL OUTPUT

```
C = 1; RBF gamma = 1 / (features x their variance), a common default; digits by one-vs-rest
Breast cancer: linear 0.9754, polynomial degree 3 0.9509, RBF 0.9813
  RBF minus linear +0.0058; better on 4, worse on 1 of 5
Digits: linear 0.9633, polynomial degree 3 0.9866, RBF 0.9814
  RBF minus linear +0.0182; better on 5, worse on 0 of 5
for scale: on Day 107 a random forest scored 0.9712 on digits and 0.9596 on breast cancer, over
    its own splits
Actual output from this lesson's run.
```

What it means

On digits the kernels win clearly: RBF 0.9814 and polynomial 0.9866 against linear 0.9633, the RBF better on 5 of 5 splits. On breast cancer the RBF gain is small (+0.0058) and the cubic kernel is worse (0.9509). Pixels interact (a stroke is several pixels together), and a curved boundary can use that; breast cancer's 30 measurements are already nearly linearly separable.

WHY CHOOSE THIS?

Large gains when the classes really curve.

WHY NOT THIS? (TRADEOFFS)

No gain, extra settings and extra cost when they do not.

5. gamma and C together

DEFINITION — start here, no prior knowledge assumed

The two settings interact: a wiggly boundary (large γ) needs less pressure (C) to fit. A grid of both, scored by 3-fold cross-validation on the training part. The grid was first γ 0.001 to 0.3; the best was at its edge, so 1 and 3 were added.

REAL-WORLD SCENARIO

Tuning bass and treble together: the best bass depends on the treble.

ACTUAL OUTPUT

```
CV accuracy        C 0.1       C 1      C 10     C 100
gamma 0.001       0.9006    0.9006    0.9149    0.9491
gamma 0.003       0.9006    0.9006    0.9388    0.9610
gamma 0.01        0.9054    0.9165    0.9579    0.9706
gamma 0.03        0.9165    0.9499    0.9777    0.9825
gamma 0.1         0.9316    0.9785    0.9889    0.9889
gamma 0.3         0.9634    0.9881    0.9905    0.9905
gamma 1           0.9793    0.9857    0.9889    0.9889
gamma 3           0.8235    0.8267    0.8148    0.8148
support share      C 0.1       C 1      C 10     C 100
gamma 0.001        0.202     0.207     0.195     0.098
gamma 0.003        0.205     0.208     0.145     0.072
gamma 0.01         0.207     0.197     0.098     0.058
gamma 0.03         0.208     0.150     0.077     0.061
gamma 0.1          0.212     0.117     0.084     0.084
gamma 0.3          0.239     0.177     0.175     0.175
gamma 1            0.841     0.839     0.840     0.840
gamma 3            0.999     1.000     1.000     1.000
best by CV: gamma 0.3, C 10; test accuracy 0.9870
Actual output from this lesson's run.
```

What it means

The best cross-validated setting is γ 0.3 with C 10 (0.9905), which scores 0.9870 on the test images. Past it, accuracy collapses: at γ 3 every image becomes a support vector (1.000) and CV accuracy falls to about 0.82. The support share is a free overfitting gauge: 0.175 at the best setting, nearly all at the worst.

WHY CHOOSE THIS?

Cross-validation finds a good region quickly.

WHY NOT THIS? (TRADEOFFS)

A wide grid of kernel SVMs is slow; each cell trains ten SVMs three times.

6. Random Fourier features

DEFINITION — start here, no prior knowledge assumed

Random features $z(x) = \sqrt{(2/D)}\cos(Wx + c)$, with W drawn from a normal distribution set by γ, give $z(x)\cdot z(x') \approx \exp(-\gamma|x - x'|^2)$. A plain linear SVM on them approximates the RBF SVM, without the n × n table.

REAL-WORLD SCENARIO

Estimating a song's similarity to others from a few hundred random audio fingerprints instead of the full recording.

ACTUAL OUTPUT

```
digits, gamma 0.03; approximation error on a 400 x 400 kernel matrix, and test accuracy of a
    LINEAR SVM on the random features (C = 1, 5 splits)
 random features  mean abs error  accuracy
              10          0.1234    0.6583
              30          0.1072    0.8686
             100          0.0469    0.9391
             300          0.0283    0.9547
            1000          0.0136    0.9581
            3000          0.0110    0.9566
the exact RBF kernel on the same splits: 0.9570
Actual output from this lesson's run.
```

What it means

The approximation error falls from 0.1234 with 10 features to 0.0110 with 3,000, and accuracy follows: 0.6583 with 10, 0.9547 with 300, matching the exact kernel (0.9570) from about 1,000. This setting is deliberately the plain default, so the comparison is with the exact kernel at the same γ, not with the best tuned score.

WHY CHOOSE THIS?

Scales to large data: memory grows with D, not with n squared.

WHY NOT THIS? (TRADEOFFS)

Random: results vary with the seed, and D in the thousands is needed for accuracy.

Interview preparation — technical questions

Q: What is the kernel trick?

A: Using K(x, z) = phi(x).phi(z) in place of dot products, so an algorithm that only needs dot products works in the feature space without building it.

Q: What does gamma do in an RBF kernel?

A: It sets how quickly similarity fades with distance: large gamma gives local, wiggly boundaries.

Q: How do you know a function is a valid kernel?

A: Its kernel matrices must be positive semi-definite; tanh(x.z - 1) failed with 62 negative eigenvalues.

Q: How do you handle ten classes with SVMs?

A: One-vs-rest: ten binary SVMs, the most confident wins.

Q: How do you scale a kernel SVM to millions of rows?

A: Approximate the kernel with explicit random features and train a linear model.

Interview preparation — theoretical questions

Q: Why does the kernel trick need the dual?

A: The dual depends on the data only through dot products; the primal needs w, which lives in the feature space.

Q: Why is the RBF feature space infinite?

A: Expanding exp(2 gamma x.z) gives a polynomial of every degree.

Q: When does a kernel not help?

A: When the classes are nearly linearly separable: breast cancer gained only +0.0058.

Q: What does the support-vector share tell you?

A: A high share means the model is memorising: 1.000 at gamma 3.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> The kernel trick swaps a dot product for a similarity, and a linear method becomes a non-linear one with the same solver. It helps a great deal when the classes curve (digits) and little when they do not (breast cancer). Its weakness is the n × n table; random features remove that at a small cost in accuracy.

- Polynomial kernel = explicit features to 5.7e-14; degree 5 would need 11,238,513 features.
- Digits: RBF 0.9814 against linear 0.9633 (5 of 5 splits); tuned 0.9870.
- Breast cancer: RBF gains +0.0058; on two features a straight line is as good.
- 3,000 random features: kernel error 0.0110, accuracy 0.9566 against the exact 0.9570.

Suggested watch order, if starting from zero

- 1. Support Vector Machines Part 2: The Polynomial Kernel (StatQuest) — how a polynomial kernel lifts points into more dimensions.
- 2. Support Vector Machines Part 3: The Radial (RBF) Kernel (StatQuest) — the RBF kernel and why its feature space is infinite.

Day 114 preview

- **Naive Bayes from scratch**, on real text: a model that counts words, assumes they are independent, and is still hard to beat on small data.